

Tema 8

Construcción de guiones. Programación

1. [Introducción](#)
 2. [Rutinas: procedimientos y funciones](#)
 3. [Variables](#)
 4. [Instrucciones condicionales](#)
 5. [Instrucciones repetitivas](#)
 6. [Funciones de librerías básicas disponibles](#)
 7. [Gestión de errores](#)
 8. [Gestión de rutinas almacenadas](#)
- [Reconocimiento](#)

1. Introducción

Casi todos los grandes Sistemas Gestores de Datos incorporan utilidades que permiten ampliar el lenguaje SQL para producir pequeñas utilidades que añaden al SQL mejoras de la programación estructurada (bucles, condiciones, funciones...). La razón es que hay diversas acciones en la base de datos para las que SQL no es suficiente. Por ello, las bases de datos incorporan algún lenguaje de tipo procedimental que permite manipular de forma más avanzada los datos de la base de datos.

En algunos SGBD, como MariaDB-MySQL, este lenguaje procedimental está incrustado en la propia estructura de lenguaje SQL, mientras en otros se trata más como una extensión. En Oracle tenemos PL/SQL, en SQL Server se utiliza Transact SQL...

Algunas de las funciones que pueden realizar estas extensiones procedimentales son:

- Facilitar la realización de tareas administrativas sobre la base de datos (copia de valores antiguos, auditorías, control de usuarios...).
- Validación y verificación avanzada de usuarios.
- Consultas muy avanzadas.
- Tareas imposibles de realizar con SQL, como, por ejemplo, establecer que unas determinadas acciones se repitan un número establecido de veces.

2. Rutinas: procedimientos y funciones

Una rutina es un conjunto de sentencias y de instrucciones de procedimiento (estructuras de control, declaración de variables...) que realiza una tarea específica y que se guarda con un nombre en la base de datos.

Una vez almacenadas, los clientes no necesitan lanzar cada comando individual que la compone, sino que, en su lugar, pueden llamar a la rutina como un único comando.

Utilizar rutinas presenta numerosas ventajas:

- Mejora el rendimiento, ya que el código se almacena en la base de datos y hay menos intercambio entre el cliente y el servidor. Las rutinas MariaDB se compilan bajo demanda para después ponerse en caché (MariaDB mantiene su propia caché de rutinas almacenadas para cada conexión). Así, si una aplicación utiliza una rutina almacenada varias veces en una sola conexión, se usa la versión compilada; y, de lo contrario, la rutina almacenada funciona como una consulta.
- Reutilización de código. La llamada a la rutina hace que se evite tener que implementar nuevamente toda la lógica interna que tiene.
- Son seguras, al permitir al administrador de la base de datos otorgar los permisos adecuados a las aplicaciones que acceden a las rutinas almacenadas en la base de datos, sin necesidad de conceder ningún permiso en las tablas de la base de datos subyacente.

Se diferencian 2 tipos de rutinas:

- Procedimientos almacenados.
- Funciones.

Un procedimiento almacenado es un conjunto de comandos SQL que pueden ser almacenados en el servidor. Al realizarse, los clientes no necesitan realizar los comandos individuales, ya que, en su lugar, pueden reutilizar los ya almacenados.

Por otro lado, tenemos las funciones, que, al igual que los procedimientos almacenados, están formadas por un conjunto de comandos SQL y están almacenados en la base de datos.

Ambos parecen ser similares, ¿cuándo y porqué usar un procedimiento y cuándo una función? La diferencia más general entre procedimientos y funciones es que cada una es llamada de modo diferente y para propósitos distintos:

- Un procedimiento no devuelve ningún valor. En su lugar, es llamado con una declaración CALL para realizar una operación, como puede ser la de modificar una tabla o procesar la recuperación de registros.
- Una función es llamada dentro de una expresión y nos devuelve (RETURNS) un valor único directamente al que lo llama para ser utilizado en la expresión.
- No se puede invocar una función con una instrucción CALL, ni se puede invocar un procedimiento en una expresión.

Para crear una rutina es necesario el privilegio *CREATE ROUTINE*, además de los propios a los objetos (tablas, vistas...) manipulados por la rutina. Para modificarla o eliminarla es necesario el *ALTER ROUTINE*. Para ejecutar una rutina, el usuario debe tener el privilegio *EXECUTE* sobre la rutina en cuestión.

La sintaxis básica para la creación de un procedimiento sería:

```
CREATE PROCEDURE nombre_procedimiento ([parámetro[...]])  
[características]  
Cuerpo del procedimiento
```

La sintaxis básica para la creación de una función sería:

```
CREATE FUNCTION nombre_funcion ([parámetro[,...]])
RETURNS tipo_variable
[características]
Cuerpo de la función
```

Por defecto, la rutina se asocia con la base de datos actual. Para asociarla con otra base de datos, habría que especificar el nombre como *nombre_bd.nombre_rutina*.

La lista de parámetros entre paréntesis debe estar siempre presente; si no hay parámetros, se debe usar una lista de parámetros vacía ().

En los procedimientos se pueden usar parámetros *IN* (entrada), *OUT* (salida) ó *INOUT* (entrada/salida); especificándolo delante del nombre del parámetro. Si no se especifica nada, se toma por defecto el valor *IN*.

[IN | OUT | INOUT] nombre_parámetro tipo_parámetro

- *IN*: permite proporcionar un valor a la rutina invocada. Para dar un valor al parámetro, la rutina que invoca puede utilizar cualquier expresión (variable, expresión literal...). Si el procedimiento almacenado o la función modifica el valor del parámetro, la modificación no es visible en la rutina que invoca al procedimiento o función.
- *OUT*: permite al procedimiento almacenado devolver un valor a la rutina que invoca. Esta rutina debe utilizar obligatoriamente una variable en la llamada para recuperar el valor devuelto.
- *INOUT*: el procedimiento recibe el parámetro y puede consultarlo y modificarlo.

En las funciones sólo se utilizan parámetros de tipo *IN* (no es necesario especificarlo, se sobreentiende).

Las *características* pueden tomar los siguientes valores:

```
LANGUAGE SQL
| [NOT] DETERMINISTIC
| { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
| SQL SECURITY { DEFINER | INVOKER }
| COMMENT 'string'
```

- *LANGUAGE SQL*: indica que la rutina está escrita en SQL.
- *[NOT] DETERMINISTIC*. Una rutina se considera determinista si siempre produce el mismo resultado para los mismos parámetros de entrada, y no determinista en cualquier otro caso. Por defecto, toma el valor no determinista. Se aplica sólo para las funciones, con procedimientos no tiene efecto.
- *{CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA}*
 - *CONTAINS SQL*: indica que la rutina sólo tiene sentencias SQL que no leen ni modifican datos. Es el valor por defecto si no se especifica ningún valor de este grupo.
 - *NO SQL*: indica que la rutina no contiene sentencias SQL.
 - *READS SQL DATA*: indica que la rutina contiene sentencias SQL que leen datos, pero no que los modifican.

- *MODIFIES SQL DATA*: indica que la rutina contiene sentencias SQL que pueden modificar datos como *INSERT*, *UPDATE* o *DELETE*. De manera implícita, también es posible contener sentencias SQL que leen datos.
- *SQL SECURITY {DEFINER | INVOKER}*: se utiliza para especificar si la rutina debe ser ejecutada usando los permisos del usuario que crea la rutina o el usuario que la invoca. Por defecto es *DEFINER*. Para poder ejecutar una rutina, es necesario tener el permiso *EXECUTE*.

El cuerpo de una rutina suele estar formado por sentencias SQL. En caso de haber más de una, deben ir dentro de un bloque delimitado por sentencias *BEGIN* y *END*.

También se pueden utilizar estas sentencias *BEGIN... END* en ámbitos fuera del cuerpo de un procedimiento o función. Cuando se utilice en este sentido habría que marcar este bloque con la cláusula *NOT ATOMIC*, es decir, se utilizaría la sintaxis:

```
[begin_label:] BEGIN [NOT ATOMIC]
    [statement_list]
END [end_label]
```

Existe una palabra reservada, *DELIMITER*, que indica el carácter de comienzo y fin de la rutina. Típicamente sería un “;”, pero dado que se necesita un “;” para cada sentencia SQL que exista dentro de la rutina, es conveniente usar otro carácter (normalmente \$\$ ó //).

Ejemplo de rutina, de tipo procedimiento, para imprimir por pantalla una cadena:

```
DELIMITER $$
DROP PROCEDURE IF EXISTS test.hola_mundo$$
CREATE PROCEDURE test.hola_mundo()
BEGIN
    SELECT 'hola mundo';
END $$
```

```
CALL test.hola_mundo()
```

```
hola
mundo
hola mundo
```

Como se puede observar en el ejemplo previo, para visualizar datos por pantalla, en MariaDB se utiliza una sentencia *SELECT*. Podríamos componer, con todos los datos que necesitemos mostrar por pantalla, una única cadena utilizando la función *CONCAT*. Si la llamada de la función devolviera más de una columna, se devolvería el error 1241. Si la llamada de la función devolviera más de una fila, se devolvería el error 1242. Por lo tanto, es conveniente utilizar una cláusula *LIMIT* para asegurarse de que solo se devuelve una fila.

Ejemplo de rutina, en este caso una función, que recibe un valor de estado como entrada y lo comprueba. Según cual sea, se asignará un valor u otro, con el comando *SET*, a la variable que devuelve la función:

```
DELIMITER $$
```

```

CREATE FUNCTION estado (in_estado CHAR(1))
    RETURNS VARCHAR(20)
BEGIN
    DECLARE estado VARCHAR(20);
    IF in_estado = 'P' THEN
        SET estado='caducado';
    ELSEIF in_estado = 'O' THEN
        SET estado='activo';
    ELSEIF in_estado = 'N' THEN
        SET estado='nuevo';
    END IF;
    RETURN (estado);
END; $$

```

```

SELECT estado('O');

```

estado('O')
activo

Veamos otro ejemplo de función y procedimiento donde uno llama a la otra.
-- Función que recibe un número y devuelve TRUE si es par y FALSE si es impar

```

DELIMITER $$
CREATE FUNCTION esimpar (numero int)
    RETURNS int
BEGIN
    DECLARE impar INT;
    IF MOD (numero,2)=0 THEN SET impar=FALSE;
    ELSE SET impar=TRUE;
    END IF;
    RETURN (impar);
END; $$

```

-- Procedimiento que utiliza la función "esimpar". Muestra un mensaje por pantalla indicándonos la paridad del parámetro que recibe como entrada

```

DELIMITER $$
DROP PROCEDURE IF EXISTS muestra_estado $$
CREATE PROCEDURE muestra_estado (IN numero int)
BEGIN
    IF (esimpar(numero)) THEN
        SELECT CONCAT (numero, ' es impar');
    ELSE
        SELECT CONCAT (numero, ' es par');
    END IF;
END; $$

```

```

CALL muestra_estado (12);

```

CONCAT (numero, ' es par')
12 es par

Más ejemplos:

Ejemplo 1:

```

CREATE FUNCTION max_salario (dept TINYTEXT) RETURNS INT RETURN

```

```
(SELECT MAX(salario) FROM emple WHERE emple.dept_no = dept);
```

```
SELECT MAX_SALARIO(10);
```

max_salario(10)
3000

Ejemplo 2: – Procedimiento que realiza consultas sobre tablas de la bd

```
DELIMITER |
```

```
CREATE PROCEDURE Listados()
```

```
BEGIN
```

```
    SELECT * FROM articulos;
```

```
    SELECT * FROM fabricantes;
```

```
END |
```

```
DELIMITER ;
```

```
CALL Listados();
```

ARTICULO	COD_FABRICANTE	PESO	CATEGORIA	PRECIO_VENTA	PRECIO_COSTO	EXISTENCIAS
Macarrones	20	1	Segunda	1.00	1.00	120
Tallarines	20	2	Segunda	1.20	1.02	100
Tallarines	20	1	Segunda	0.99	0.51	100

...

COD_FABRICANTE	NOMBRE	PAIS
10	CALVO	ESPAÑA
15	LU	BELGICA

...

3. Variables

En primer lugar, merece la pena destacar la diferencia entre las variables definidas por el usuario y las variables locales.

Las variables definidas por el usuario son variables creadas por el usuario y existen mientras dure la sesión. No pueden ser accesibles por otros usuarios y expiran cuando la sesión se cierra. Sin embargo, estas variables pueden ser compartidas entre varias consultas y programas almacenados.

Estas variables van precedidas de un carácter @, sus nombres no distinguen entre mayúsculas y minúsculas, no son declaradas, pueden ser leídas incluso si aún no se les ha asignado ningún valor (se leería el valor NULL).

Para asignar valor a este tipo de variables se puede utilizar:

- Sentencia SET.
 - Sintaxis: `SET @variable=valor;` ó `SET @variable:=valor`

Ejemplo:

```
SET @Variable_marca="SEAT";
```

```
SET @Variable_marca:="SEAT"
```

- Operador := en una sentencia SQL.
 - Si se utiliza sólo el símbolo = en vez de := entonces se estaría evaluando dicha expresión, es decir, se trata de una comparación; devolvería 1 si es verdadero y 0 si es falso.

Ejemplo:

```
SELECT Matricula, Marca, Modelo FROM VEHICULOS
WHERE marca=@Variable_marca; -- Operador de comparación '='
```

Estas variables se almacenan en la tabla `USER_VARIABLES` dentro de `information_schema`, guardando su nombre, su valor, su tipo y el conjunto de caracteres que fija.

Ejemplos:

```
SET @userVariable=10;
SET @userVariable2='HOLA';
SELECT @userVariable3 := '1979/02/16';
```

```
SELECT * FROM information_schema.USER_VARIABLES ORDER BY
VARIABLE_NAME;
```

VARIABLE_NAME	VARIABLE_VALUE	VARIABLE_TYPE	CHARACTER_SET_NAME
userVariable	10	INT	utf8mb4
userVariable2	HOLA	VARCHAR	utf8mb4
userVariable3	1979/02/16	VARCHAR	utf8mb4

```
SELECT @userVariable3 = 'HOLA';
-- En este caso, se está evaluando si @userVariable3 tiene el valor 'HOLA'.
```

```
@userVariable3 =
'HOLA'
0
```

Por otro lado, las variables locales se utilizan en procedimientos almacenados, funciones... No van precedidas del prefijo @ y necesitan ser declaradas anteponiéndolas la palabra reservada `DECLARE`. El valor se especifica como una expresión (incluso se permiten subconsultas). Los nombres de estas variables no distinguen entre mayúsculas y minúsculas.

El ámbito de una variable local es el bloque `BEGIN ... END` donde es declarada. Se puede hacer referencia a la variable en bloques anidados dentro del bloque donde se la declara, con la excepción de aquellos bloques que declaran una variable con el mismo nombre, que serán dos variables diferentes.

Deben ser declaradas antes de `CONDITIONS`, `CURSORs` y `HANDLERs`.

La sintaxis para su declaración es:

```
DECLARE var_name [, var_name] ... [[ROW] TYPE OF]] type [DEFAULT value]
```

Se les puede proporcionar un valor por defecto, para lo que habría que incluir la cláusula `DEFAULT`. Si no hay cláusula `DEFAULT`, el valor inicial es `NULL`.

Ejemplo:

```
DECLARE var2 INT DEFAULT 1;-- Declaración dentro de un procedimiento/función
```

Se puede definir el tipo de dato de una variable según el tipo de otro objeto base en lugar de establecerlo específicamente en la declaración. De esta manera, si el tipo de objeto base cambia, también lo hará el tipo de dato de la variable cuyo tipo de asoció. Para ello utilizamos la cláusula *[ROW] TYPE OF*.

Ejemplo:

```
DECLARE v_numEmple TYPE OF emple.emp_no
-- 'v_numEmple' será de mismo tipo que el tipo que
-- la columna 'emp_no' de la tabla 'emple'
```

```
DECLARE v_e ROW TYPE OF emple;
-- 'v_e' será el mismo tipo de datos de fila que la tabla 'emple'; es decir, podremos
-- acceder a v_e.emp_no, v_e.apellido, v_e.oficio, v_e.salario...
```

La principal diferencia entre variables locales y variables definidas por el usuario es que las variables locales son reiniciadas con el valor *NULL* cada vez que el procedimiento almacenado (o función) donde se utiliza es llamado, mientras que las variables de sesión no son reiniciadas con *NULL*.

Ejemplo de uso de ambos tipos de variables:

```
DELIMITER //
CREATE PROCEDURE sp_LocalAndUserDefinedVariableDemo()
BEGIN
  DECLARE localVariable INT DEFAULT 10;
  SET localVariable=localVariable+10;
  SET @userVariable=@userVariable+10;
  SELECT localVariable;
  SELECT @userVariable;
END;//
```

```
DELIMITER ;
SET @userVariable=10;
```

-- 1ª llamada al procedimiento

```
CALL sp_LocalAndUserDefinedVariableDemo();
```

localVariable	@userVariable
20	20

-- Nueva llamada al procedimiento

```
CALL sp_LocalAndUserDefinedVariableDemo();
```

localVariable	@userVariable
20	30

Como podemos comprobar en el ejemplo, la variable local se reinicia con cada llamada al procedimiento, mientras que la variable de sesión se mantiene durante toda la sesión.

Al igual que en otros lenguajes de programación, los procedimientos y funciones utilizan variables y parámetros que determinan la salida del algoritmo. Los tipos de variables y parámetros son los usuales de MariaDB.

Ejemplo: Se recibe un valor entero por parámetro, se declaran 2 variables (variable1 y variable2) de tipo entero y se testea el valor del parámetro recibido. Si es 17, se asigna su valor a la variable1; si no, se le asigna el valor 30 a la variable2.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS proc1 $$

CREATE PROCEDURE proc1 /*nombre*/
(IN parametro1 INTEGER) /*parámetros*/
BEGIN
    DECLARE variable1 INTEGER; /*variable*/
    DECLARE variable2 INTEGER; /*variable*/
    IF parametro1=17 THEN
        SET variable1 = parametro1; /*asignación*/
    ELSE
        SET variable2 = 30; /*asignación*/
    END IF;
    INSERT INTO t VALUES (variable1, variable2); /*instrucción SQL*/
END $$
```

Como se ha comentado anteriormente, los procedimientos pueden devolver resultados a través de los parámetros de tipo *OUT* (y lógicamente, *INOUT*). Para ello, se almacenan los resultados de una consulta directamente en variables utilizando la sentencia:

```
SELECT NombreColumnas [, ...] INTO NombreVariables [, ...]
```

De esta manera, dejamos los valores devueltos en las variables indicadas en lugar de mostrarlos en pantalla. Para hacer esto, la consulta debe devolver una sola fila ya que no se puede asignar a una variable una lista de valores.

Como hemos visto en los ejemplos anteriores, también se puede utilizar la cláusula *SET* para asignar valor a una variable de este tipo, utilizando '=' ó ':='.

Ejemplo:

```
SET Variable_marca="SEAT"; -- ó SET Variable_marca:="SEAT"
```

Cuando un procedimiento utiliza un parámetro de salida, para llamar al procedimiento es necesario pasar una variable que cargue el dato devuelto por el procedimiento:

```
CALL NombreProcedimiento(@Num);
```

Una vez cargada, esa variable @Num se puede usar en cualquier sentencia, como, por ejemplo:

```
SELECT @Num;
```

Ejemplos:

Ejemplo 1:

/* Se crea un procedimiento que recibe como parámetro de entrada la matrícula de un vehículo y devuelve en el parámetro de salida el número de reparaciones que ha sufrido ese vehículo. */

```
DELIMITER |
```

```
CREATE PROCEDURE NumReparaciones (IN Matri varchar(8), OUT NumRep
INT)
```

```

BEGIN
    SELECT COUNT(Matricula) INTO NumRep
    FROM REPARACIONES
    WHERE Matricula=Matri;
END |

DELIMITER ;
-- Para ejecutar el procedimiento:
CALL NumReparaciones('1313 DEF', @NumRep);

-- Para ver el valor devuelto:
SELECT @NumRep AS 'Numero de reparaciones';

-- Utilizando el procedimiento anterior, se podría obtener un listado de los
vehículos que igualen o superen el número de reparaciones anterior ejecutando
una consulta como...
SELECT VEHICULOS.Matricula, Marca, Modelo, Count(*)
FROM VEHICULOS INNER JOIN REPARACIONES ON
VEHICULOS.Matricula=REPARACIONES.Matricula
GROUP BY VEHICULOS.Matricula
HAVING Count(*) >= @NumRep;

```

Ejemplo 2:

/* Imprimir un mensaje por pantalla mostrando el salario modificado del empleado con código 7499. Se almacenarán 2 valores en variables utilizando la cláusula SELECT... INTO */

```

DELIMITER $$
CREATE PROCEDURE s_Emplea2()
BEGIN
    DECLARE v_salario DECIMAL(9,2);
    DECLARE v_apellido VARCHAR(50);
    SELECT ROUND(salario*1.2,2), UPPER(apellido) INTO v_salario,
v_apellido FROM emple WHERE emp_no=7499;
    SELECT CONCAT('El nuevo salario de ', V_APELLIDO, ' será de ',
V_SALARIO,' euros');
END$$

```

```

DELIMITER ;
CALL s_Emplea2();

```

```
concat('El nuevo salario de ', V_APELLIDO, '
será de ', V_SALARIO,' euros')
```

```
El nuevo salario de ARROYO será de 1800.00 e...
```

Como resumen de la asignación de valores a variables, veamos el siguiente ejemplo:

```

DELIMITER //
CREATE OR REPLACE PROCEDURE sp_LocalAndUserDefinedVariableDemo()
BEGIN
    DECLARE localVariable INT DEFAULT 10;
    -- 1.- Asignación valores con SET, da igual usar = que :=
    SET localVariable=localVariable+10;
    SET @userVariable=@userVariable+10;

```

```

SET localVariable:=100;
SET @userVariable:=100;

-- 2.- Uso de operadores = y := con SELECT
SELECT localVariable=200; -- Se trata de una comparación:
    -- ¿es localVariable igual a 200? --> Imprime 0 (100 no es igual a 200)
SELECT @userVariable=200; -- Se trata de una comparación:
    -- ¿es @userVariable igual a 200? --> Imprime 0 (100 no es igual a 200)
-- SELECT localVariable:= 300; -- ERROR
SELECT @userVariable:=300; -- Se trata de una asignación de valor
    -- Imprime 300

-- 3.- Uso de SELECT ... INTO
SELECT 400 INTO localVariable; -- Asignación de valor; no imprime nada
SELECT 400 INTO @userVariable; -- Asignación de valor; no imprime nada

-- Impresión de estos últimos valores
SELECT localVariable;
SELECT @userVariable;
END;

DELIMITER ;
SET @userVariable:=10;
CALL sp_LocalAndUserDefinedVariableDemo();

```

*** EJERCICIOS – Realiza los ejercicios 1-5 ***

4. Instrucciones condicionales

En MariaDB se pueden utilizar las siguientes instrucciones condicionales:

- *IF* – De tipo simple, es decir, cuando sólo hay una condición.
- *IF THEN ELSE* – De tipo múltiple, cuando hay dos o más posibilidades.
- *CASE* – Cuando tenemos un conjunto de condiciones distintas.

IF – IF THEN ELSE

La sintaxis para *IF-THEN-ELSE* es:

```

IF search_condition THEN statement_list
    [ELSEIF search_condition THEN statement_list] ...
    [ELSE statement_list]
END IF;

```

Ejemplos:

Ejemplo 1:

```

DELIMITER |
DROP PROCEDURE IF EXISTS InsertarIncluyen |
CREATE PROCEDURE InsertarIncluyen (IN IdRec char(10), IN IdRepar INT , IN
Unid SMALLINT)
BEGIN
    /*Obtiene las unidades en stock de ese recambio */
    DECLARE Num INT;
    SELECT Stock INTO Num FROM RECAMBIOS WHERE IdRecambio = IdRec;

```

```

/*Si hay stock suficiente, se actualiza*/
IF Num >= Unid THEN
    INSERT INTO Incluyen VALUES (IdRec, IdRepar, Unid);
    UPDATE RECAMBIOS SET Stock=Stock-Unid;
END IF;
END |
DELIMITER ;

```

Ejemplo 2:

```

DELIMITER //
CREATE FUNCTION PageCount ( value INT )
RETURNS varchar(10) DETERMINISTIC
BEGIN
    DECLARE level varchar(20);

    IF value < 500 THEN
        SET level = 'Bajo';
    ELSEIF value >= 500 AND value <= 4000 THEN
        SET level = 'Medio';
    ELSE
        SET level = 'Alto';
    END IF;

    RETURN level;
END //

DELIMITER ;
SELECT PageCount(230);

```

PageCount(230)
Bajo

CASE

La sintaxis de la instrucción CASE puede ser de 2 opciones:

1)

```

CASE case_value
    WHEN when_value THEN statement_list
    [WHEN when_value THEN statement_list] ...
    [ELSE statement_list]
END CASE

```

2)

```

CASE
    WHEN search_condition THEN statement_list
    [WHEN search_condition THEN statement_list] ...
    [ELSE statement_list]
END CASE

```

Ejemplos:

Ejemplo 1:

```

DELIMITER //

```

```

CREATE FUNCTION CalcLevel ( value INT )
RETURNS varchar(20) DETERMINISTIC
BEGIN
    DECLARE level varchar(20);
    CASE value
        WHEN 4000 THEN
            SET level = 'Bajo';
        WHEN 5000 THEN
            SET level = 'Medio';
        ELSE
            SET level = 'Otro';
    END CASE;

    RETURN level;
END; //

```

```

DELIMITER ;
SELECT CalcLevel (4500);

```

CalcLevel (4500)
Otro

Ejemplo 2:
DELIMITER //

```

CREATE FUNCTION CalcLevel2 ( value INT )
RETURNS varchar(20) DETERMINISTIC
BEGIN
    DECLARE level varchar(20);
    CASE
        WHEN value <= 4000 THEN
            SET level = 'Low';
        WHEN value > 4000 AND value <= 5000 THEN
            SET level = 'Medium';
        ELSE
            SET level = 'Other';
    END CASE;
    RETURN level;
END; //

```

```

DELIMITER ;
SELECT CalcLevel2 (4500);

```

CalcLevel2 (4500)
Medium

*** EJERCICIOS – Realiza los ejercicios 6-11 ***

5. Instrucciones repetitivas

MariaDB permite el uso de instrucciones repetitivas con las cláusulas:

- *LOOP*.
- *FOR*.
- *REPEAT... UNTIL*
- *WHILE*.

Además, cuenta con 3 cláusulas asociadas al uso de estas instrucciones:

- *LEAVE*, se utiliza cuando se quiere salir de un bloque de código identificado por una *etiqueta*, tales como bucles *LOOP*, *WHILE* y *REPEAT*.
- *RETURN*, generalmente se utiliza cuando se quiere salir de una función, para devolver el resultado de la función. También se utilizar para terminar un bucle *LOOP* y salir de él con un resultado.
- *ITERATE*, se utiliza cuando se quiere ejecutar de nuevo un bloque repetitivo. Se usa con bucles *LOOP*, *WHILE* y *REPEAT*.

LOOP

LOOP se utiliza cuando queremos que el bucle se ejecute al menos una vez. Se puede salir de un bucle *LOOP* tanto con la cláusula *LEAVE* como con *RETURN*.

La sintaxis de *LOOP* es:

```
[begin_label:] LOOP
    statement_list
END LOOP [end_label]
```

La *etiqueta* permite especificar un nombre al *LOOP* para podernos referir a él dentro del bloque.

Ejemplos:

-- Ejemplo 1: Repetir un bucle mientras un valor sea menor que 850.

DELIMITER //

CREATE FUNCTION CalcValueLoop (starting_value INT)

RETURNS INT DETERMINISTIC

BEGIN

DECLARE total_value INT;

SET total_value = 0;

label1: LOOP

SET total_value = total_value + starting_value;

IF total_value < 850 THEN

ITERATE label1;

END IF;

LEAVE label1;

END LOOP label1;

RETURN total_value;

END; //

-- Ejemplo 2: Igual ejemplo que el anterior, pero utilizando como condición de salida del bucle *LOOP* la cláusula *RETURN*.

DELIMITER //

CREATE FUNCTION CalcValueLoopRepeat (starting_value INT)

RETURNS INT DETERMINISTIC

```

BEGIN
    DECLARE total_value INT;
    SET total_value = 0;
    label1: LOOP
        SET total_value = total_value + starting_value;
        IF total_value < 3000 THEN
            ITERATE label1;
        END IF;
        RETURN total_value;
    END LOOP label1;
    RETURN starting_value;
END; //
```

/ En este caso, el primer RETURN permitirá la salida del bucle LOOP, saldrá de la función y devolverá el valor almacenado en la variable total_value.
El segundo RETURN se utiliza al final de la función para salir de la misma y devolver el valor almacenado en la variable starting_value.
Es decir, se están creando 2 posibles puntos de salida de la función.*/*

FOR

Los bucles *FOR* se han introducido a partir de la versión 10.3 de MariaDB. Son muy utilizados cuando sabemos el número fijo de veces que se tiene que ejecutar un código.

La sintaxis de este bucle es:

```

[begin_label:]
FOR var_name IN [ REVERSE ] lower_bound .. upper_bound
DO statement_list
END FOR [ end_label ]
```

Si se especifica la opción *REVERSE*, el contador se irá decrementando en cada iteración.

Ejemplos:

-- Ejemplo 1:

```

DELIMITER //
FOR i IN 1..3          -- Puede parecer que hay error en el índice, pero ejecuta bien
DO
    INSERT INTO t1 VALUES (i);
END FOR;
//
```

```

DELIMITER ;
SELECT * FROM t1;
```

a
1
2
3

-- Ejemplo 2:

```

CREATE OR REPLACE TABLE t1 (a INT);
DELIMITER //
FOR i IN REVERSE 4..12
```

```

DO
  INSERT INTO t1 VALUES (i);
END FOR;
//

```

```

DELIMITER ;
SELECT * FROM t1;

```

a
12
11
10
9
8
7
6
5
4

REPEAT ... UNTIL

REPEAT... UNTIL, como tiene la condición de finalización al final, siempre entrará al menos una vez en el bucle.

La sintaxis de *REPEAT ... UNTIL* es:

```

[begin_label:] REPEAT
  statement_list
UNTIL search_condition
END REPEAT [end_label]

```

Ejemplo:

```

DELIMITER //
CREATE FUNCTION CalcValueRepeat ( starting_value INT )
RETURNS INT DETERMINISTIC
BEGIN
  DECLARE total_value INT;
  SET total_value = 0;
  label1: REPEAT
    SET total_value = total_value + starting_value;
  UNTIL total_value >= 999
  END REPEAT label1;
  RETURN total_value;
END; //

```

WHILE

Las sentencias encerradas en un bucle *WHILE* se repiten mientras la condición del bucle sea verdadera.

La sintaxis de *WHILE* es:

```

[begin_label:] WHILE search_condition DO
  statement_list
END WHILE [end_label]

```

Al tener la condición del bucle al principio del mismo, puede ocurrir que no llegue a entrar nunca en el mismo.

Ejemplos:

Ejemplo 1:

```
DELIMITER //
CREATE FUNCTION CalcValueWhile ( starting_value INT )
RETURNS INT DETERMINISTIC
BEGIN
    DECLARE total_value INT;
    SET total_value = 0;
    label1: WHILE total_value <= 999 DO
        SET total_value = total_value + starting_value;
    END WHILE label1;
    RETURN total_value;
END; //
```

Ejemplo 2: Realizar un procedimiento que carga en una tabla, con una única columna F de tipo DATE, las fechas en las que no se han realizado reparaciones de coches, desde la fecha pasada como parámetro hasta la fecha actual.

-- Se crea la tabla TablaFechas

```
CREATE TABLE TablaFechas (Fecha1 DATE);
```

```
DELIMITER |
```

```
DROP PROCEDURE IF EXISTS InsertarFechas |
```

```
CREATE PROCEDURE InsertarFechas(IN fecha DATE)
```

```
BEGIN
```

```
    DECLARE num_reparaciones INT;
```

```
    /*el bucle se repetirá desde la fecha introducida hasta la actual*/
```

```
    WHILE fecha < curdate() DO
```

```
        /*obtenemos cuántas reparaciones ha habido
        en la fecha a analizar*/
```

```
        SELECT count(*) INTO num_reparaciones
        FROM REPARACIONES WHERE FechaEntrada=fecha;
```

```
        /*si no ha habido reparaciones, se inserta la fecha en la tabla*/
```

```
        IF num_reparaciones=0 THEN
```

```
            INSERT INTO TablaFechas values(fecha);
```

```
        END IF;
```

```
        /*en cada iteración se actualiza la fecha al día siguiente*/
```

```
        SET fecha=adddate(fecha, INTERVAL 1 DAY);
```

```
    END WHILE;
```

```
END |
```

```
CALL InsertarFechas('2019-03-16');
```

```
SELECT * FROM tablafechas;
```

*** EJERCICIOS – Realiza los ejercicios 12-17 ***

6. Funciones de librería básicas disponibles

Todo SGBD permite que los usuarios creen sus propias funciones y procedimientos. Además, los SGBD también incluyen un conjunto de funciones predefinidas y propias de cada SGBD.

Normalmente las funciones operan con las columnas de una tabla, se utilizan en las expresiones que indican los datos que se muestran en una consulta, en las condiciones de la cláusula WHERE, en las expresiones SET para obtener los datos con los que se modifica una columna, en los valores que se insertan mediante INSERT, etc.

MariaDB incluye un numeroso conjunto de funciones. Como ya vimos en un tema previo, las funciones se pueden clasificar en:

- Funciones numéricas.
- Funciones de cadenas.
- Funciones de trabajo con nulos.
- Funciones de fecha y hora.
- Funciones de conversión.
- Funciones de control de flujo.
- Funciones de listas.
- Funciones de agrupación o de totales.
- Otras funciones.

Se pueden consultar detalladamente en la documentación oficial de MariaDB:

<https://mariadb.com/kb/en/built-in-functions/>

Ejemplo: Crear una función para calcular los días que pasa un vehículo en el taller de reparaciones, pasándole la fecha de entrada y la de salida. Además, se utiliza una función propia del SGBD, como *datediff*.

```
DELIMITER |
CREATE FUNCTION dias (FechaE date, FechaS date)
RETURNS INT
BEGIN

    DECLARE NDIAS INT;
    SET NDIAS=datediff(FechaS,FechaE);
    RETURN NDIAS;

END |

-- Posteriormente, podemos utilizar nuestra función dentro de un SELECT
DELIMITER ;
SELECT IdReparacion, Matricula, dias (FechaEntrada, FechaSalida) AS 'Dias en el taller' FROM reparaciones WHERE matricula='1313 DEF';
```

*** EJERCICIOS – Realiza los ejercicios 18-21 ***

7. Gestión de errores

La gestión de los errores de ejecución se lleva a cabo mediante condiciones y manejadores (*handler*).

Una condición es, en realidad, una forma de llamar a los errores específicos. Una condición puede declararse con la sintaxis:

```
DECLARE condition_name CONDITION FOR condition_value

condition_value:
    SQLSTATE [VALUE] sqlstate_value
| mysql_error_code
```

Una condición puede asociarse a un código de error MariaDB (numérico) o a un código de error SQL (cadena de 5 caracteres, cláusula *SQLSTATE*). Si la sentencia no ha generado ningún error, entonces devuelve 0 como código de ejecución correcto.

Si una sentencia forma parte de un procedimiento o función y genera un error, entonces se terminaría automáticamente la ejecución del procedimiento o función. Esto, generalmente puede ser un problema ya que, aunque una sentencia de una rutina produzca un error, puede que se desee procesar las siguientes sentencias de la rutina. Para solucionar este problema, MariaDB permite usar manipuladores de errores o handlers que sirven para indicar cómo debe responder el servidor MariaDB, en procedimientos y funciones, ante situaciones de error.

Para declarar un manipulador de error hay que usar la sintaxis:

```
DECLARE handler_type HANDLER
    FOR condition_value [, condition_value] ...
    statement

handler_type:
    CONTINUE
| EXIT

condition_value:
    SQLSTATE [VALUE] sqlstate_value
| condition_name
| SQLWARNING
| NOT FOUND
| SQLEXCEPTION
| mariadb_error_code
```

Si ocurre alguna de las condiciones asociadas al handler, entonces se ejecutan las sentencias especificadas. Si se especifica más de una sentencia, hay que escribirlas en un bloque *BEGIN... END*.

Los handler se declaran después de las variables locales.

Hay 2 tipos de handler:

- *CONTINUE*. En este tipo de handler, la ejecución del programa actual continúa después de la ejecución de las sentencias del handler.
- *EXIT*. A diferencia del anterior, la ejecución de la rutina termina al acabar de ejecutarse las sentencias del propio handler.

Los valores de condiciones pueden tomar diferentes especificaciones:

- *SQLSTATE*. Representa un valor *SQLSTATE* (literal de cadena de 5 caracteres) o un código de error de MariaDB (número).
- *Condition_name*. Nombre de la condición previamente especificado con *DECLARE... CONDITION* en el programa almacenado.
- *SQLWARNING*. Abreviatura de la clase de valores *SQLSTATE* que comienzan por '01'.
- *NOT FOUND*. Abreviatura de la clase de valores *SQLSTATE* que comienzan por '02'.
- *SQLEXCEPTION*. Abreviatura de la clase de valores *SQLSTATE* que comienzan por '00', '01' y '02'.
- *Mariadb_error_code*. Código de error propio de MariaDB.

El valor *SQLSTATE* '00000' o el código de error 0 representan éxito, por lo que no deberían utilizarse como códigos de error propiamente dicho.

En la Web <https://mariadb.com/kb/en/mariadb-error-codes/> hay una descripción completa de los códigos de error utilizados en MariaDB. Algunos ejemplos podemos visualizar en la siguiente tabla:

Código de error	SQLSTATE	Error	Descripción
1005	HY000	ER_CANT_CREATE_TABLE	No se puede crear la tabla '%s'
1006	HY000	ER_CANT_CREATE_DB	No se puede crear la base de datos '%s'
1022	23000	ER_DUP_KEY	No se puede escribir; clave duplicada en la tabla '%s'
1052	23000	ER_NON_UNIQ_ERROR	La columna '%s' en %s es ambigua
1060	42S21	ER_DUP_FIELDNAME	Nombre de columna '%s' duplicado
1065	42000	ER_EMPTY_QUERY	Consulta vacía
1111	HY000	ER_INVALID_GROUP_FUNC_USE	Uso inválido de una función de grupo
1125	HY000	ER_UDF_EXISTS	La función '%s' ya existe
1141	42000	ER_NONEXISTING_GRANT	No hay tal privilegio definido para el usuario '%s' en el host '%s'
1149	42000	ER_SYNTAX_ERROR	Error de sintaxis
1217	23000	ER_ROW_IS_REFERENCED	No se puede borrar o actualizar una fila padre: una clave foránea falla
1239	42000	ER_WRONG_FK_DEF	Incorrecta definición de clave foránea '%s':%s

Ejemplo: Supongamos que tenemos el siguiente procedimiento...

```
DELIMITER |
CREATE PROCEDURE Manipuladores()
BEGIN
    INSERT INTO VEHICULOS (Matricula, Marca, Modelo) VALUES
('1234BMY','audi','A6');
    SELECT * FROM CLIENTES;
    SELECT COUNT(*) FROM CLIENTES;
    SELECT * FROM REPARACIONES;
END |
DELIMITER ;
```

/ Suponiendo que hacemos la llamada al procedimiento y la matrícula a insertar ya existe en la tabla, no se ejecutará ninguna de las sentencias que siguen a la sentencia INSERT que da error*/*

```
CALL Manipuladores()
ERROR 1062 (23000): Duplicate entry '1234BMY' for key 1
```

/ Sabiendo que un error de inserción por clave duplicada da el valor de estado 23000 y el código de error 1062, modificaríamos el procedimiento anterior para que controlase ese error de una forma similar a la siguiente... */*

```
DELIMITER |
CREATE PROCEDURE Manipuladores()
BEGIN
    DECLARE CONTINUE HANDLER FOR SQLSTATE '23000' SELECT 'Se ha
producido un error';

    INSERT INTO VEHICULOS (Matricula,Marca, Modelo) VALUES ('2233
ABC','Ford','Mondeo');

    SELECT * FROM CLIENTES;
    SELECT COUNT(*) FROM CLIENTES;
    SELECT * FROM REPARACIONES;
END|
DELIMITER ;
```

*/*Ahora, al ejecutar el procedimiento, se ejecuta la sentencia asociada al manipulador cuando se produce el error, es decir, se escribe el mensaje indicando que se ha producido un error y se continua (tipo CONTINUE) con la sentencia que hay a continuación de la sentencia INSERT que produjo error. Si el manipulador fuese tipo EXIT se ejecutaría la sentencia asociada al manipulador, pero no las siguientes y se iría al final del procedimiento.*/*

*/*La declaración del manipulador podríamos haberla hecho usando el código de error y no el valor de estado de la sentencia. En su lugar podríamos haber puesto: DECLARE CONTINUE HANDLER FOR 1062 SELECT 'Se ha producido un error';*/*

Cuando se utilizan bloques anidados, los errores se propagan de la siguiente manera:

- Si se produce un error en un bloque anidado y este error se gestiona en el bloque anidado, la ejecución puede continuar, bien en el bloque anidado (instrucción *CONTINUE*), bien en el bloque principal (instrucción *EXIT*).
- Si se produce un error en un bloque anidado y este error no se gestiona en el bloque anidado, la ejecución del bloque anidado termina (*EXIT* de manera predeterminada) y MariaDB busca un manejador para el error en el bloque principal. Si no hay error, la ejecución del bloque principal también se termina (*EXIT* de manera predeterminada); si hay uno, se ejecuta y la ejecución continúa o no en el bloque principal según el tipo de instrucción (*CONTINUE* o *EXIT*).

También se pueden provocar errores de manera explícita a través de la instrucción *SIGNAL*. Se utiliza para personalizar errores por el propio usuario. La sintaxis es:

```
SIGNAL error_condition
    [SET error_property
    [, error_property] ...]

error_condition:
    SQLSTATE [VALUE] 'sqlstate_value'
    | condition_name

error_property:
    error_property_name = <error_property_value>

error_property_name:
    CLASS_ORIGIN
    | SUBCLASS_ORIGIN
    | MESSAGE_TEXT
    | MYSQL_ERRNO
    | CONSTRAINT_CATALOG
    | CONSTRAINT_SCHEMA
    | CONSTRAINT_NAME
    | CATALOG_NAME
    | SCHEMA_NAME
    | TABLE_NAME
    | COLUMN_NAME
    | CURSOR_NAME
```

error_condition puede ser un valor *SQLSTATE* o un nombre de condición definido en la sección *DECLARE CONDITION*. Para errores personalizados, se recomienda usar el código *SQLSTATE* 45000.

Se puede consultar más información de este mecanismo en la web oficial de MariaDB: <https://mariadb.com/kb/en/signal/>.

Ejemplo:

```
CREATE PROCEDURE test_error(n INT)
BEGIN
    DECLARE 'too_big' CONDITION FOR SQLSTATE '45000';
```

```

IF n > 10 THEN
    SIGNAL 'too_big';
END IF;
END;

```

*** EJERCICIOS – Realiza los ejercicios 22-23 ***

8. Gestión de rutinas almacenadas

Las rutinas (procedimientos almacenados y funciones) se manipulan con los comandos *CREATE*, *ALTER*, *DROP* y *SHOW*.

La orden *CREATE* ya la hemos visto en los apartados anteriores.

Para cambiar las características de un procedimiento o de una función, se puede utilizar la sentencia *ALTER*, cuya sintaxis es:

ALTER {PROCEDURE | FUNCTION} nombre [Características...]

Ejemplo: *ALTER PROCEDURE simpleproc SQL SECURITY INVOKER;*

Para eliminar procedimientos o funciones hemos usado, en los ejemplos previos, el comando *DROP*, cuya sintaxis es:

DROP {PROCEDURE | FUNCTION} [IF EXISTS] nombre_rutina;

Además, se puede ver información de las rutinas creadas utilizando el comando *SHOW*.

Para devolver la cadena exacta que corresponde a la creación de la rutina:

SHOW CREATE {PROCEDURE | FUNCTION} nombre_rutina;

Ejemplo:

SHOW CREATE FUNCTION CalcValueWhile;

Function	sql_mode	Create Function	character_set_client	collation_connection	Database Collation
CalcValueWhile	NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGI...	CREATE DEFINER = 'root'@'localhost' FUNCTION `CalcValueWhile` (starting...	utf8mb4	utf8mb4_general_ci	utf8mb4_spanish_ci

Si se quiere hacer algún tipo de filtro por patrones y devolver información como el nombre de la base de datos, el tipo de rutina, el creador, la fecha de creación, etc; se puede usar la orden:

SHOW {PROCEDURE | FUNCTION} STATUS [LIKE 'patrón']

Si no se incluye patrón, la información se muestra para todos los procedimientos/funciones.

Ejemplo:

-- Para ver características funciones que empiecen por C.

SHOW FUNCTION STATUS LIKE 'C%';

Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection	Database Collation
bd_tema8	CalcLevel	FUNCTION	root@localhost	2021-03-11 13:40:18	2021-03-11 13:40:18	DEFINER		utf8mb4	utf8mb4_general_ci	utf8mb4_spanish_ci
bd_tema8	CalcLevel2	FUNCTION	root@localhost	2021-03-11 13:42:59	2021-03-11 13:42:59	DEFINER		utf8mb4	utf8mb4_general_ci	utf8mb4_spanish_ci
bd_tema8	CalcValueLoop	FUNCTION	root@localhost	2021-03-11 16:23:17	2021-03-11 16:23:17	DEFINER		utf8mb4	utf8mb4_general_ci	utf8mb4_spanish_ci
bd_tema8	CalcValueRepeat	FUNCTION	root@localhost	2021-03-11 16:36:58	2021-03-11 16:36:58	DEFINER		utf8mb4	utf8mb4_general_ci	utf8mb4_spanish_ci
bd_tema8	CalcValueWhile	FUNCTION	root@localhost	2021-03-11 17:29:10	2021-03-11 17:29:10	DEFINER		utf8mb4	utf8mb4_general_ci	utf8mb4_spanish_ci

La orden *SHOW* se nutre del diccionario de datos gracias a la tabla *INFORMATION_SCHEMA.ROUTINES*, que también se puede consultar con instrucciones *SELECT*.

Bibliografía/Webgrafía:

- <https://mariadb.com/kb/en/documentation>
- <https://www.techonthenet.com/mariadb/index.php>
- <https://www.mysqltutorial.org/mysql-stored-procedure-tutorial.aspx>
- <https://docplayer.es/3943942-Tema-8-construccion-de-guiones-de-administracion.html>
- https://ikastaroak.birt.eus/edu/argitalpen/backupa/20200331/1920k/es/ASIR/GBD/GBD06/es_ASIR_GBD06_Contenidos/website_index.html
- <https://blog.mdcloud.es/procedimientos-mysql-ventajas-desventajas/>
- <https://informaticaynadamas.com/2020/04/25/mysql-funciones-y-procedimientos-almacenados/>
- <https://www.tutorialspoint.com/user-defined-variables-vs-local-variables-in-mysql>
- PHP y MySQL. Domine el desarrollo de un sitio Web dinámico e interactivo (4ª edición). Editorial ENI. Autor: Olivier Heurtel.